UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

Taller 10: versión inicial integrada de los componentes de software definitivos

Sistema escalable de seguridad residencial (Sentinel Home)

Hamed Valdez Jason Alejandro

Matrícula 1290897

Proyecto de Carrera

Docente: José Isabel García Rocha

21 de octubre de 2026

Repositorio: https://github.com/HamedJason/ProyectoDeCarrera

Versión descrita: rama principal, último commit 26551bc (más los cambios de este taller)

1. Resumen y cumplimiento de los requisitos

Este documento entrega la versión inicial integrada del software del proyecto. Todo lo descrito existe en el repositorio y corre como una sola cadena: el nodo C3 lee los sensores, el concentrador ESP32 decide la alarma y la envía al backend, el backend valida y guarda en PostgreSQL, y la aplicación web (PWA) muestra el dato actual y el historial. Se suma el módulo de cámara, que el Taller 8 dejaba como punto abierto y que ya quedó resuelto con una cámara MC210 conectada por un puente ESP32-S3.

Criterio de honestidad. Cada requisito lleva un estado: Cumple (existe y se verificó), Cumple con evidencia por anexar (existe, pero la captura o el resultado debe tomarse en el equipo real) o Parcial. Las pruebas automáticas se ejecutaron al preparar este documento; las pruebas con hardware y las capturas se listan en el Anexo A para que se agreguen antes de entregar.

Área	Requisito del taller y dónde se cumple	Estado
General	Estructura organizada del repositorio (sección 2).	Cumple
General	Diagrama actualizado de módulos y responsabilidades (sección 3).	Cumple
General	Pseudocódigo o diagramas de flujo de los procesos principales (sección 4).	Cumple
Firmware	Código que compile sin errores: firmware/nodo_sensor, nodo_c3 y camara_ov5640 (sección 5). Se compila con el IDE de Arduino y las librerías de la sección 9.	Cumple con evidencia por anexar (captura de compilación)
Firmware	Lectura de entradas reales: contactos MC-38 y PIR AM312 en el C3.	Cumple
Firmware	Envío por el protocolo definitivo: ESP-NOW (nodo a concentrador) y HTTP/JSON con token (concentrador a backend).	Cumple
Firmware	Salida o acción central: sirena en GPIO 4 con etapa MOSFET (diseñada y simulada en el Taller 9).	Cumple
Firmware	Formato de mensajes definido: paquete de 81 bytes y JSON (sección 5.2).	Cumple
Firmware	Estrategia básica de reconexión: Wi-Fi no bloqueante, cola local de 20 eventos, 3 reintentos de alarma.	Cumple
Backend	Servicio desplegado: Docker Compose en un VPS, dominio sentinelhome.ddns.net, HTTPS por proxy y puerto 8080.	Cumple con evidencia por anexar (respuesta de /salud)
Backend	Mínimo dos operaciones, registro y consulta: POST /mediciones y GET /mediciones (más más de 40 endpoints).	Cumple
Backend	Validación de datos reales, conexión a la base de datos y manejo de errores 400, 401, 403, 404, 429 y 500.	Cumple
Backend	Documentación breve de las operaciones (sección 6, tabla de operaciones).	Cumple
Base de datos	Diagrama entidad-relación, diccionario de datos, base en el servicio definitivo y procedimiento reproducible (sección 7).	Cumple
Base de datos	Datos reales de prueba y operaciones de registro y consulta.	Cumple con evidencia por anexar

Área	Requisito del taller y dónde se cumple	Estado
		(consulta sobre la base real)
Frontend	Bosquejo de pantallas y varias pantallas implementadas con comunicación real con el backend (sección 8).	Cumple
Frontend	Dato actual e historial; estados de carga, vacío y error de comunicación.	Cumple
Calidad	Tecnologías y versiones, variables de entorno sin secretos, instrucciones de compilación y plan de pruebas con al menos tres pruebas (sección 9).	Cumple

2. Estructura del repositorio

El repositorio separa las capas por carpeta, y cada una puede leerse y desplegarse sin las demás. Los secretos no viven en el repositorio: .gitignore excluye todos los archivos .env y la búsqueda de claves y tokens sobre los archivos versionados no encontró coincidencias.

```
ProyectoDeCarrera/
├── README.md, PASOS.md          Visión general y guía de puesta en marcha
├── firmware/
│   ├── nodo_c3/                 ESP32-C3: contactos MC-38 y PIR, emite por ESP-NOW
│   ├── nodo_sensor/             ESP32 concentrador: alarma, sirena, HTTP
│   ├── camara_ov5640/           Cámara propia (ESP32-S3 + OV5640)
│   └── puente_camara_s3/        Puente de la cámara MC210 (NAT + WireGuard)
├── backend/
│   ├── server.js, auth.js, camara.js     API REST, cuentas y cámara
│   ├── eventos_camara.js, onvif/         Detección de persona y movimiento por ONVIF
│   ├── camara_rtsp.js, camara_medir.js   Fotos/vivo con ffmpeg y medición
│   ├── db.js, schema.sql, consultas.sql  Conexión, estructura y consultas
│   ├── publico/                          PWA (index.html, sw.js, manifest.json)
│   └── test/                             Pruebas automáticas (node --test)
├── deploy/                      Compose, Caddyfile, desplegar_vps.sh, .env.example
├── frontend/                    React Native del Taller 5 (sustituido por la PWA)
├── pruebas/                     simular.js, medir_tiempos.js, PRUEBAS.md
├── proteus/                     Taller 9: circuito y simulación
└── docs/                        Talleres 1 a 10 y diagramas con sus scripts
```

Nota sobre frontend/. La carpeta contiene la prueba de concepto en React Native del Taller 5. El frontend definitivo es la aplicación web progresiva que sirve el backend desde backend/publico, porque funciona en iPhone y Android sin tienda de aplicaciones y recibe notificaciones push. La carpeta se conserva como antecedente.

3. Diagrama de módulos y responsabilidades

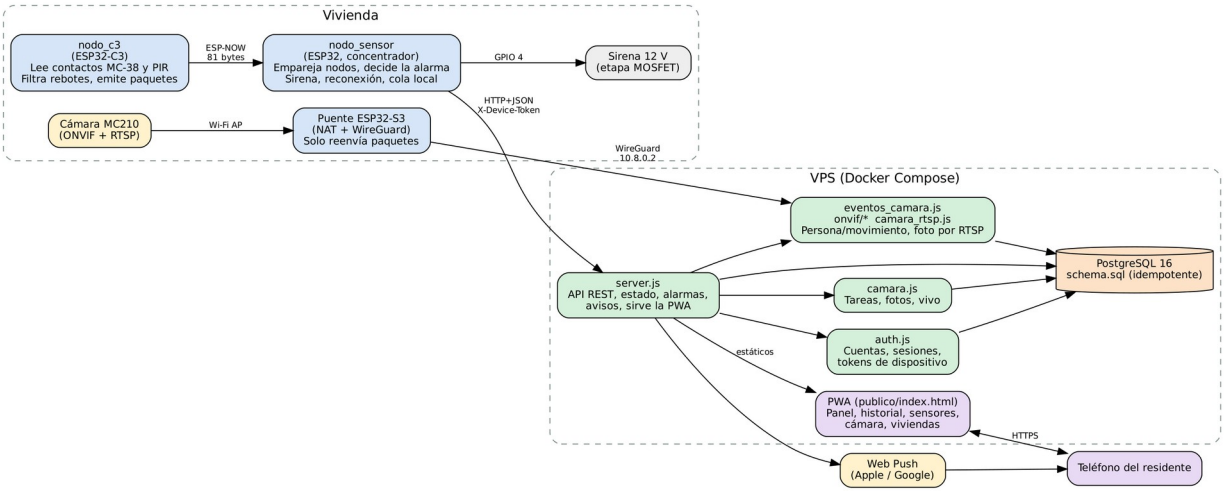


Figura 1. Módulos del sistema, su ubicación y los protocolos entre ellos.

Módulo	Responsabilidad (y lo que NO hace)	Estado
nodo_c3	Lee los contactos y el PIR, filtra rebotes (50 ms) y bloquea el PIR 8 s entre eventos. Se empareja con el concentrador y emite eventos y un estado cada 30 s. No habla con Internet.	Implementado
nodo_sensor (concentrador)	Recibe los paquetes, decide la alarma con el estado armado y activa la sirena sin depender de la red. Informa al backend, guarda eventos si no hay Wi-Fi y reconecta. Guarda Wi-Fi y token por comandos seriales.	Implementado
Puente ESP32-S3	Une la cámara con el VPS por un túnel WireGuard. Solo reenvía paquetes; no procesa video.	Implementado (pruebas de estabilidad pendientes)
server.js	Valida y registra mediciones, mantiene el estado de armado, procesa alarmas, vigila nodos y concentrador, envía avisos y sirve la PWA.	Implementado
auth.js	Cuentas (scrypt), sesiones por cookie, viviendas, miembros, invitaciones y tokens de dispositivo por vivienda.	Implementado
camara.js, camara_rtsp.js	Cola de tareas, fotos, vista en vivo con límites de tiempo, y captura por RTSP con ffmpeg solo cuando se pide.	Implementado
eventos_camara.js y onvif/*	Se suscribe a los eventos de la cámara, decide qué es un cambio real y cuándo avisar. No analiza video.	Implementado
PostgreSQL y schema.sql	Persistencia de 17 tablas; la estructura se crea sola al arrancar el backend.	Implementado
PWA (publico/)	Interfaz del residente: estado, historial, sensores, cámara y viviendas. Solo muestra y ordena; no decide la alarma.	Implementado

4. Procesos principales: diagramas de flujo y pseudocódigo

4.1 Flujo de una alarma, del contacto al teléfono

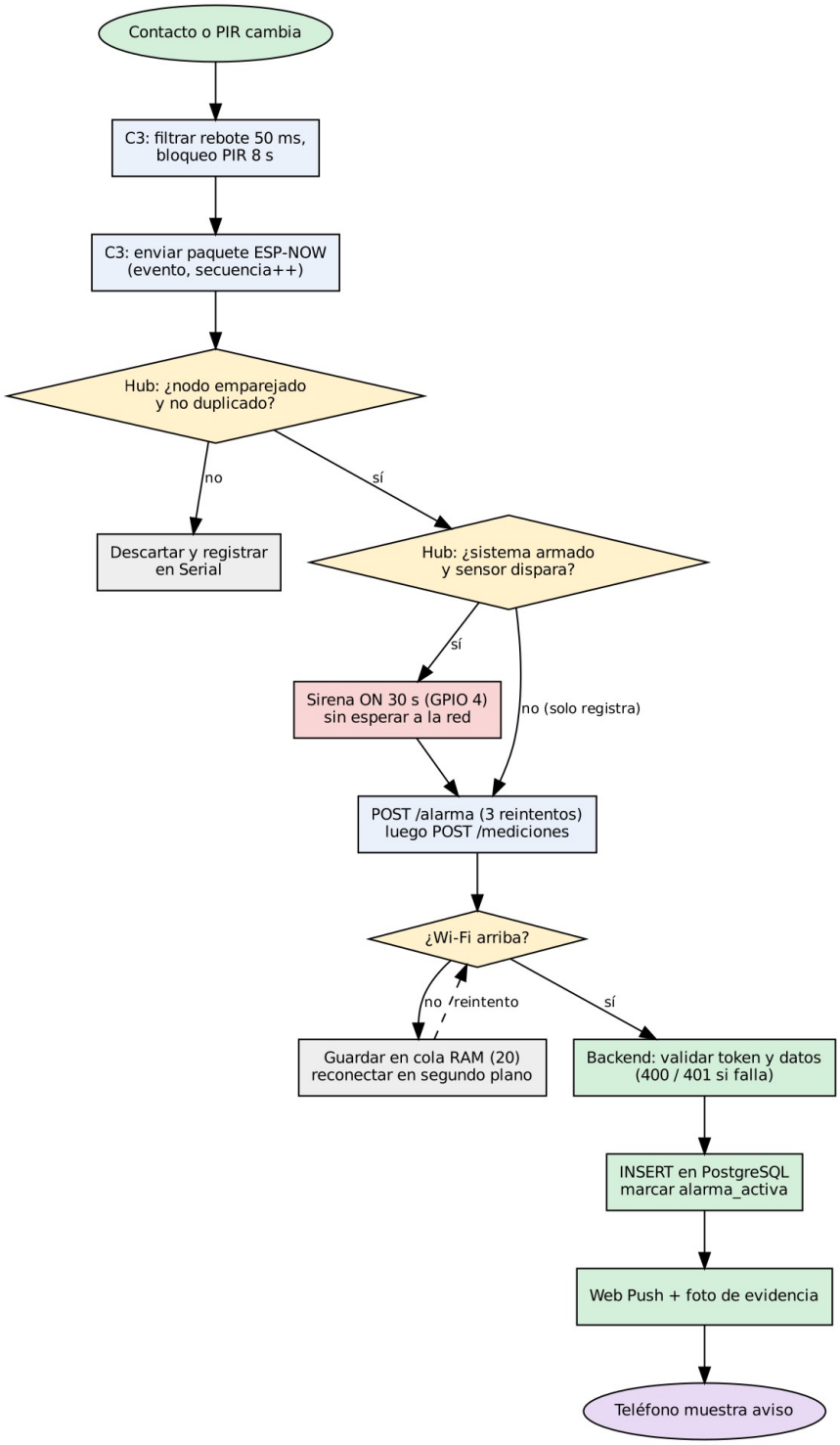


Figura 2. Flujo de una alarma. La sirena suena antes de hablar con la red.

```
CICLO DEL CONCENTRADOR (loop):
```

```
  atenderWiFi()                          // nunca bloquea; reintenta cada
INTERVALO_RECONEXION_MS
  si llegó un paquete ESP-NOW:
      si magia != "SR" o version != 1:   descartar
      si el nodo no está emparejado:     descartar y avisar por Serial
      si secuencia <= última del nodo:   descartar (duplicado)
      si tipo == EVENTO y sistema armado y no modo_silencioso:
          sirena = ENCENDIDA durante 30 s            // decisión local
          encolar POST /alarma (hasta 3 reintentos)
      encolar POST /mediciones
  cada 2 s:   GET /estado   -> actualiza armado, modo_silencioso, alarma_activa
  cada 30 s:  POST /nodos/latido
  si Wi-Fi caído: guardar hasta 20 eventos en RAM y enviarlos al reconectar
```

4.2 Flujo de la detección de la cámara

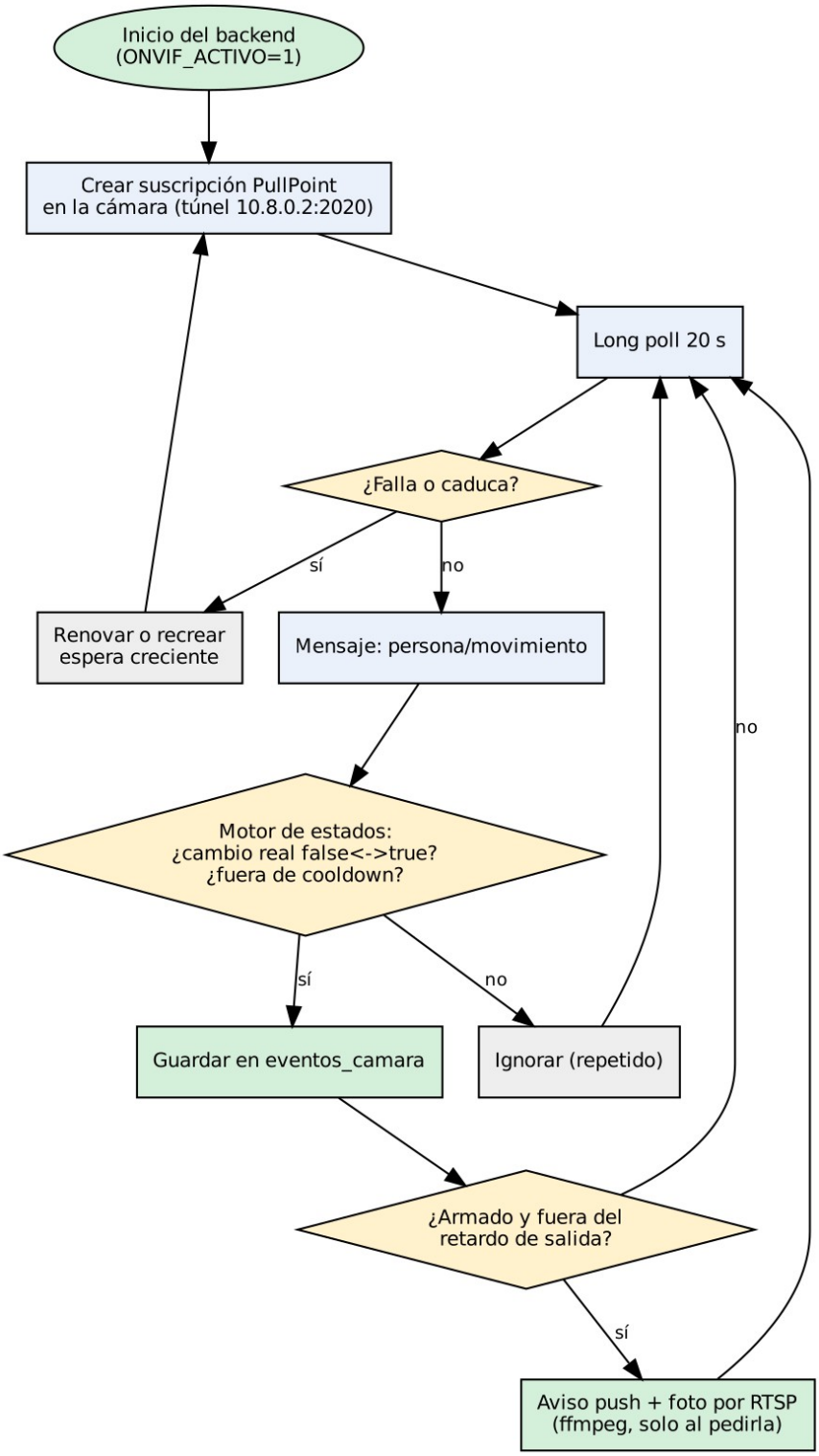


Figura 3. Detección de persona y movimiento por ONVIF y alerta con foto.

```
SERVICIO ONVIF (backend):
  crear suscripción PullPoint en la cámara (túnel 10.8.0.2:2020)
  repetir:
```

```
      mensajes = esperar eventos (long poll, 20 s)
      si falló o caducó: renovar o recrear con espera creciente; marcar cámara desconocida
      para cada mensaje:
          si es el primero tras conectar:       tomarlo como sincronización (sin alerta)
          si el valor es igual al anterior:     ignorar (repetido)
          si cambió false<->true:               guardar en eventos_camara
              si subida y sistema armado y fuera del retardo de salida y fuera del
cooldown:
                  avisar por push y pedir foto por RTSP (ffmpeg solo en ese momento)
```

4.3 Registro y consulta de una medición

```
POST /mediciones (token de dispositivo):
  vivienda = vivienda del token            // el cuerpo no puede cambiarla
  errores  = validar(cuerpo)               // obligatorios y valor numérico
  si hay errores: responder 400 con la lista de errores
  INSERT en mediciones; registrar sensor y nodo si son nuevos
  responder 201 con el registro y la hora del servidor

GET /mediciones (sesión de usuario):
  exigir que el usuario sea miembro de la vivienda   // 401 / 403
  SELECT ... ORDER BY creado_en DESC LIMIT min(limite, 200)
```

5. Firmware

5.1 Qué hace cada programa

Programa	Entrada real, salida y comunicación
nodo_c3.ino	Entradas: contactos MC-38 en GPIO 4 y 3 (pull-up, activo bajo) y PIR AM312 en GPIO 5. Salida: paquete ESP-NOW de 81 bytes. Busca el canal del concentrador (1 a 13) y guarda su MAC en memoria no volátil.
nodo_sensor.ino	Entradas: paquetes ESP-NOW. Salidas: sirena en GPIO 4 (intermitente cada 300 ms, 30 s) y LED en GPIO 2. Comunicación: HTTP/JSON al backend con X-Device-Token. Configuración por comandos seriales: wifi, token, servidor, emparejar y ayuda.
camara_ov5640.ino	Captura con el OV5640 y sube JPEG al backend a petición. Es la alternativa propia; la cámara en uso es la MC210 por el puente.

5.2 Formato de mensajes

El enlace del nodo al concentrador usa una estructura binaria de 81 bytes idéntica en ambos extremos: magia "SR" (2 bytes), versión (1), tipo (1: evento, estado, hola, emparejar, respuesta), secuencia (4), nodoId (16), sensorId (24), variable (16), zona (16) y valor (1 con signo). El enlace del concentrador al backend usa JSON:

```
POST /mediciones   (encabezado X-Device-Token: hv_...)
{ "vivienda_id": "casa-001", "zona": "entrada", "nodo_id": "c3-a1b2c3",
  "sensor_id": "sensor-puerta-01", "variable": "estado_puerta",
  "valor": 1, "numero_registro": 1042 }
```

```
201 -> { "ok": true, "mensaje": "Medicion registrada correctamente.", "medicion":
{ ... } }
400 -> { "ok": false, "mensaje": "...", "errores": [ "Falta el campo
obligatorio \"sensor_id\"." ] }
```

5.3 Estrategia de reconexión

El Wi-Fi nunca detiene el ciclo principal: mientras se espera la red, los sensores siguen leyéndose y la sirena sigue funcionando. La pérdida de conexión se detecta, los eventos se guardan en una cola de 20 elementos y se reintenta en segundo plano:

```
void atenderWiFi() {
  if (cfgSsid.length() == 0) return;
  if (WiFi.status() == WL_CONNECTED) {
    if (!wifiEstabaConectado) { /* conectado: guardar IP y sincronizar hora */ }
    return;
  }
  if (wifiEstabaConectado) {            // se acaba de perder
    Serial.println("[WiFi] Conexion perdida. Los eventos se guardaran localmente.");
    wifiEstabaConectado = false;
    tUltimoIntentoWiFi = millis();
    return;
  }
  if (millis() - tUltimoIntentoWiFi > INTERVALO_RECONEXION_MS) {
    WiFi.disconnect();
    conectarWiFi();                     // reintento sin bloquear
  }
}
```

Compilación. Los tres programas se compilan en el IDE de Arduino con el soporte de placas ESP32 (el código admite el núcleo 3.x con ESP-IDF 5 y versiones anteriores) y la librería ArduinoJson 7. En este entorno de preparación no hay compilador de Arduino, por lo que la captura de compilación exitosa se toma en la computadora del proyecto (Anexo A).

6. Backend

El backend es un servicio Node.js con Express que corre en un contenedor junto a PostgreSQL. Los dispositivos se identifican con un token por vivienda y las personas con una sesión. Los errores se devuelven siempre como JSON con el campo ok en falso y un mensaje, con el código HTTP que corresponde.

6.1 Operaciones de registro y consulta

Método	Ruta	Credencial	Función y respuestas
POST	/mediciones	Token de dispositivo	Registra un evento o estado de sensor. Campos obligatorios: vivienda_id, nodo_id, sensor_id, variable y valor numérico. 201 con el registro; 400 con la lista de errores; 401 sin token válido; 500 si falla la base.
GET	/mediciones	Sesión	Historial, de la más reciente a la más antigua. Parámetros: vivienda_id y limite (máximo 200). 401 o 403 si no

Método	Ruta	Credencial	Función y respuestas
			pertenece a la vivienda.
GET	/mediciones/ultima	Sesión	Dato más reciente de la vivienda (el "dato actual" de la aplicación).
POST	/alarma	Token de dispositivo	Marca una alarma activa o inactiva con motivo, zona y sensor; dispara la notificación y la foto.
GET / POST	/estado	Token o sesión	GET: armado, modo silencioso, salida manual y alarma. POST (sesión): armar, desarmar, silenciar o apagar la alarma.
POST	/nodos/latido	Token de dispositivo	Informa nodos y sensores vistos; los sensores nuevos se registran solos.
GET / POST / DELETE	/sensores	Sesión	Consulta, nombra, reubica y elimina sensores.
GET	/acciones	Sesión	Bitácora de acciones de los usuarios.
GET	/camara/tarea	Token de dispositivo	La cámara espera hasta 20 s por una orden de captura.
POST	/camara/foto, /camara/cuadro	Token de dispositivo	Sube una foto de alarma o un cuadro de la vista en vivo.
GET	/fotos, /camara/eventos, /camara/deteccion	Sesión	Galería, eventos de persona y movimiento, y estado del detector.
POST	/auth/registro, /entrar, /salir	Pública o sesión	Cuenta y sesión. Límite de intentos y verificación de origen.
GET	/salud	Ninguna	Disponibilidad del servicio y de la base de datos.

La lista completa de rutas (más de 40) está en server.js, auth.js y camara.js. Las que alimentan la prueba de registro y consulta de este taller son POST /mediciones y GET /mediciones.

6.2 Validación y manejo de errores

```
function validarMedicion(cuerpo) {
  const errores = [];
  if (!cuerpo || typeof cuerpo !== 'object')
    return ['El cuerpo de la peticion debe ser un objeto JSON.'];
  for (const campo of ['vivienda_id','nodo_id','sensor_id','variable','valor']) {
    if (cuerpo[campo] === undefined || cuerpo[campo] === null || cuerpo[campo] === '')
      errores.push(`Falta el campo obligatorio "${campo}".`);
  }
  if (cuerpo.valor !== undefined && cuerpo.valor !== null && cuerpo.valor !== '' &&
      Number.isNaN(Number(cuerpo.valor)))
    errores.push('El campo "valor" debe ser numerico.');
  return errores;
}
```

El endpoint toma la vivienda del token del dispositivo e ignora la que diga el cuerpo, de modo que un dispositivo no puede escribir en la vivienda de otra persona. Las consultas usan parámetros ($1, $2, …), no concatenación de texto.

6.3 Despliegue

El despliegue se hace con docker compose up -d --build desde la carpeta deploy (o con desplegar_vps.sh, que además verifica el acceso público). El servicio expone el puerto 8080 para los dispositivos y un proxy inverso entrega HTTPS a la aplicación. La base de datos solo es visible dentro de la red interna de Docker.

7. Base de datos

Se usa PostgreSQL 16 en el mismo VPS. Se eligió un modelo relacional porque las entidades principales (cuentas, viviendas, sensores, eventos) tienen relaciones claras y se consultan por vivienda y por fecha. El diagrama distingue las relaciones con clave foránea (línea continua) de las lógicas por texto vivienda_id (línea punteada), que se mantienen así a propósito para no atar los datos de los dispositivos al ciclo de vida de las cuentas.

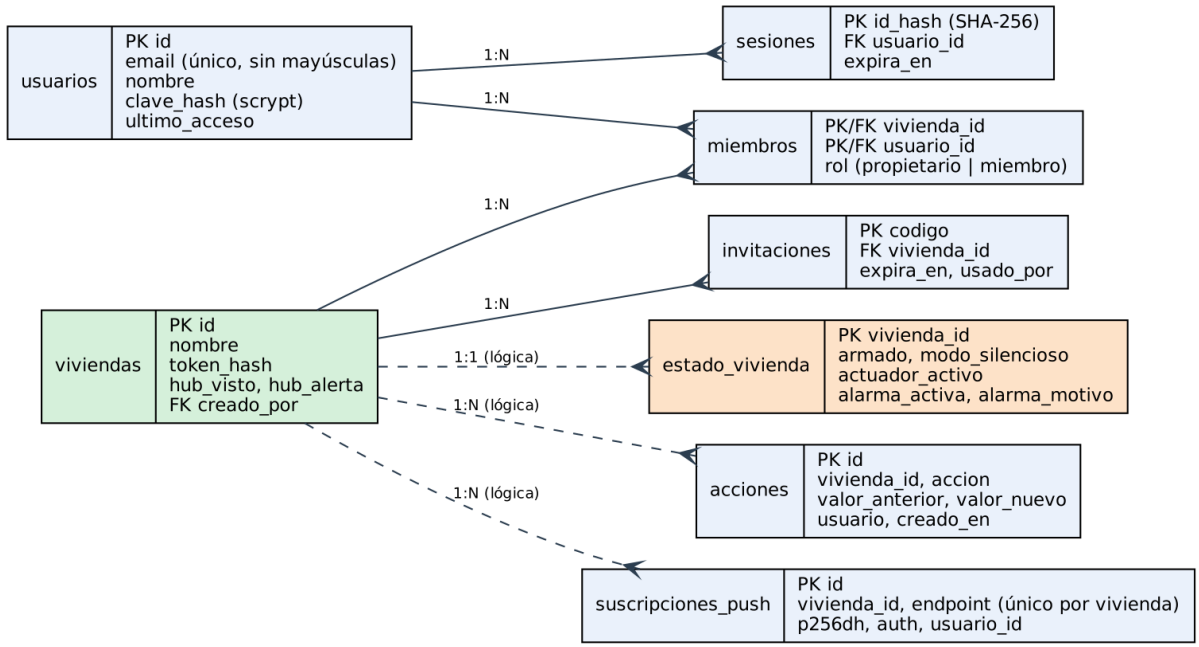


Figura 4a. Modelo entidad-relación: cuentas, viviendas, estado, bitácora y avisos.

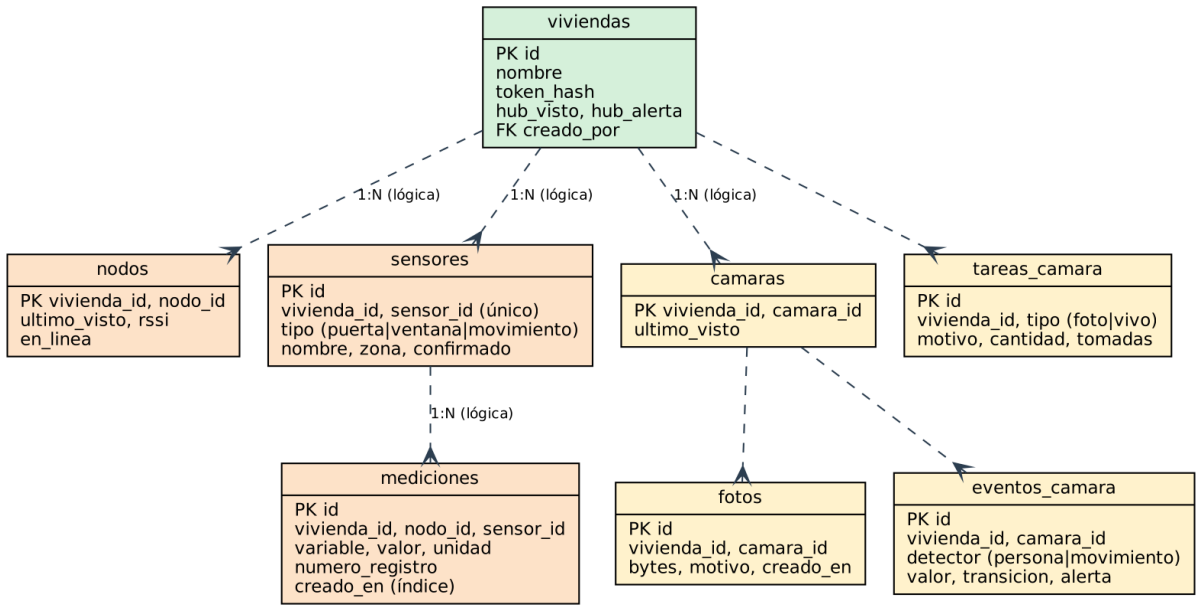


Figura 4b. Modelo entidad-relación: sensores, mediciones y cámara. Naranja: datos de sensores; amarillo: cámara.

7.1 Diccionario de las tablas principales

Tabla	Campos clave	Descripción
mediciones	id, vivienda_id, nodo_id, sensor_id, variable, valor (numérico), unidad, numero_registro, creado_en	Evento o estado de un sensor. creado_en lo pone el servidor. Índice (vivienda_id, creado_en DESC) para el historial.
estado_vivienda	vivienda_id (PK), armado, modo_silencioso, actuador_activo, alarma_activa, alarma_motivo, alarma_sensor	Estado de armado y de la alarma. Persiste para que sobreviva a un reinicio.
sensores	vivienda_id + sensor_id (único), nodo_id, tipo, nombre, zona, confirmado	Registro de sensores. tipo solo admite puerta, ventana o movimiento (restricción CHECK).
nodos	vivienda_id + nodo_id (PK), ultimo_visto, rssi, en_linea	Estado de cada nodo inalámbrico, para avisar si deja de responder.
acciones	vivienda_id, accion, valor_anterior, valor_nuevo, usuario, creado_en	Bitácora de lo que hizo cada usuario (trazabilidad).
usuarios / sesiones	email único sin distinguir mayúsculas, clave_hash (scrypt); id_hash (SHA-256 del identificador de sesión), expira_en	Cuentas y sesiones. Solo se guardan resúmenes; con la base no se pueden usar sesiones activas.
viviendas / miembros / invitaciones	viviendas.token_hash; miembros(vivienda, usuario, rol); invitaciones.codigo, expira_en	Multivivienda con propietarios y miembros. El token del dispositivo se guarda como resumen.
camaras / tareas_camara / fotos	camara_id, tipo (foto|vivo), cantidad, bytes	Cola de capturas y registro de fotos. La imagen está en disco; la base guarda solo su registro.
eventos_camara	detector (persona|movimiento), valor, transicion (subida|bajada),	Cada cambio real que reporta la cámara y si llegó a avisar.

Tabla	Campos clave	Descripción
	alerta, nota	
suscripciones_push / ajustes_servidor	endpoint, p256dh, auth; clave, valor	Dispositivos que aceptaron avisos y claves VAPID del servidor.

7.2 Procedimiento reproducible

```
# Opción A (la que usa el despliegue): se crea sola al arrancar el backend.
cd deploy && cp .env.example .env     # definir POSTGRES_PASSWORD y DEVICE_TOKEN propios
docker compose up -d --build          # db.js ejecuta backend/schema.sql

# Opción B: a mano, en cualquier PostgreSQL 14 o superior
createdb alarma
psql -d alarma -f backend/schema.sql  # idempotente: CREATE ... IF NOT EXISTS y ALTER ...
IF NOT EXISTS

# Verificar
psql -d alarma -c "\dt"              # deben verse las 17 tablas
```

El script es idempotente: puede ejecutarse sobre una base existente sin perder datos, lo que permite actualizar la base del VPS con el mismo archivo.

7.3 Datos de prueba y operaciones

Los datos de prueba se generan con el mismo protocolo que usa el hardware, desde cualquier computadora con Node 18 o superior:

```
node pruebas/simular.js --url https://sentinelhome.ddns.net --token hv_XXXX registro   #
nodo C3 con 2 contactos y un PIR
node pruebas/simular.js --url https://sentinelhome.ddns.net --token hv_XXXX alarma     #
evento de puerta y aviso

-- Consulta de verificación (backend/consultas.sql)
SELECT sensor_id, variable, valor, creado_en
FROM mediciones
WHERE vivienda_id = 'casa-001'
ORDER BY creado_en DESC LIMIT 10;
```

Registro y consulta quedan cubiertos por POST /mediciones (INSERT) y GET /mediciones (SELECT). Actualización e índices avanzados: el modelo ya incluye actualización de estado (POST /estado) y los índices de historial por vivienda y fecha, porque el proyecto los requiere.

8. Frontend

El frontend es una aplicación web progresiva en un solo archivo (backend/publico/index.html), sin dependencias externas, servida por el propio backend. Se instala en la pantalla de inicio y recibe notificaciones push mediante su service worker.

8.1 Bosquejo de las pantallas

Pantalla	Contenido	Datos del backend
Acceso	Pestañas Entrar y Crear cuenta; mensaje de primera cuenta.	POST /auth/entrar, /auth/registro
Panel	Estado de armado, botón armar o desarmar, plano con tarjetas por sensor (Abierta, Cerrada, Movimiento, Sin datos) y banner si el concentrador no se comunica.	GET /estado (cada 2.5 s), GET /sensores, GET /mediciones/ultima
Historial	Línea de tiempo de eventos con opción "Ver todo el historial".	GET /mediciones, GET /acciones
Sensores y zonas	Alta automática, nombre, tipo y zona de cada sensor.	GET, POST y DELETE /sensores
Cámara	Estado, foto bajo demanda, vista en vivo, galería y eventos de persona y movimiento.	/camara/*, /fotos
Ajustes y viviendas	Notificaciones, prueba de sirena, viviendas, miembros, invitaciones y token del dispositivo.	/viviendas, /push/*, /estado

8.2 Estados de la interfaz

- **Carga:** la lista de eventos muestra "Consultando el servidor" mientras llega la respuesta.
- **Datos vacíos:** "Todavía no hay eventos. Abre una puerta o pasa frente a un sensor para generar el primero." y "Todavía no hay sensores", con instrucciones de qué hacer.
- **Error de comunicación:** "No hay comunicación con el servidor. Puede estar apagado o sin conexión a Internet." y el indicador del pie cambia a "Sin conexión".
- **Dato actual e historial:** cada tarjeta muestra el último valor (Abierta, Cerrada, Sin datos) y el historial se pide al backend con GET /mediciones.

Las capturas de pantalla de estos estados se anexan en el Anexo A. La adaptación completa a dispositivos y las validaciones avanzadas se completan en la versión beta, como indica el taller.

9. Documentación y calidad

9.1 Tecnologías y versiones principales

Componente	Versión	Uso
Node.js	20 en el contenedor (se probó también con 22)	Backend
Express / pg	express ^4.19.2, pg ^8.11.5	API y acceso a PostgreSQL
web-push / fast-xml-parser / cors / dotenv	^3.6.7 / ^4.5.1 / ^2.8.5 / ^16.4.5	Avisos, ONVIF, CORS y variables
PostgreSQL	16 (imagen postgres:16-alpine)	Base de datos
Docker Compose y proxy inverso con HTTPS	Ver deploy/docker-compose.yml	Despliegue y certificado

Componente	Versión	Uso
ffmpeg	Instalado en el contenedor del backend	Fotos y vista en vivo por RTSP
WireGuard y esp32_nat_router (martin-ger)	Firmware oficial precompilado	Túnel de la cámara
IDE de Arduino, soporte ESP32, ArduinoJson	Núcleo ESP32 3.x (IDF 5) o anterior; ArduinoJson 7.x	Firmware
Pruebas	node --test, @electric-sql/pglite ^0.5.10	Pruebas automáticas con una base SQL real en memoria

9.2 Variables de entorno (sin secretos)

Los valores reales viven en deploy/.env, que está excluido de Git. La plantilla deploy/.env.example solo trae valores de ejemplo.

Variable	Por omisión	Para qué sirve
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	(definir)	Acceso a la base. La contraseña es propia y nunca se versiona.
DEVICE_TOKEN	(definir)	Token global heredado; se prefiere el token por vivienda (hv_…).
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT	Se generan solas	Firma de las notificaciones push.
REGISTRO_ABIERTO, MODO_PRUEBAS	falso	Permiten altas abiertas y borrar historial de pruebas.
FOTOS_*, VIVO_*, CAMARA_VIVO_FPS	Ver .env.example	Topes de espacio y tiempo y cuadros por segundo de la vista en vivo.
ONVIF_ACTIVO, ONVIF_VIVIENDA_ID, ONVIF_HOST, ONVIF_PUERTO	0, vacío, 10.8.0.2, 2020	Encienden y dirigen la detección de la cámara.
ONVIF_USUARIO / ONVIF_CLAVE	vacío	Cuenta de la cámara. Solo en el .env del servidor.
ONVIF_ALERTAR, ONVIF_SIRENA, ONVIF_COOLDOWN_*, ONVIF_RETARDO_SALIDA_S	persona, vacío, 60 y 120 s, 60 s	Política de avisos y de sirena por detección.
Wi-Fi y token del firmware	Memoria no volátil	Se escriben por comandos seriales; el código no contiene valores.

9.3 Compilar y ejecutar

```
# Backend en local
cd backend && npm install && cp .env.example .env && npm start

# Despliegue completo (base de datos, backend y proxy)
cd deploy && cp .env.example .env && docker compose up -d --build
```

```
curl -s https://sentinelhome.ddns.net/salud       # debe responder ok: true

# Firmware (IDE de Arduino)
#  1. Instalar ArduinoJson 7.x  2. Placa "ESP32 Dev Module" (concentrador) o ESP32-C3
#  3. Cargar y abrir el monitor serie a 115200
#  4. Configurar por comandos:  wifi <ssid> <clave>   token <hv_...>   servidor <url>
emparejar

# Pruebas automáticas
cd backend && npm test
```

9.4 Plan de pruebas

ID	Prueba	Procedimiento y resultado esperado	Tipo	Estado
PP-01	Registro válido	POST /mediciones con los cinco campos obligatorios. Espera 201 y una fila nueva en mediciones.	Simulador	Por anexar captura
PP-02	Dato inválido	POST /mediciones sin sensor_id y con valor "abc". Espera 400 con la lista de errores y ninguna fila nueva.	Manual / curl	Por ejecutar
PP-03	Token incorrecto	Petición de dispositivo con token inventado. Espera 401 sin efecto en la base.	Automática	Cumple (e2e.test.js)
PP-04	Consulta e historial	GET /mediciones con sesión de un miembro: lista ordenada, máximo 200. Sin sesión: 401; de otra vivienda: 403.	Automática y manual	Parcial: 401 cubierto
PP-05	Reconexión del concentrador	Apagar el router 2 minutos con el sistema armado y abrir un contacto. La sirena suena; al volver el Wi-Fi llegan los eventos guardados.	Hardware	Por ejecutar
PP-06	Detección de la cámara	Motor de estados: sincronización, transiciones, repetidos, cooldown y reconexión; servicio ONVIF contra una cámara simulada.	Automática	Cumple (onvif_estados, onvif_servicio, eventos_camara)
PP-07	Flujo completo	Contacto abierto con la casa armada: alarma en la app, push y foto en menos de unos segundos (medir_tiempos.js).	Hardware	Por anexar resultado

Resultado de la ejecución automática. Al preparar este documento se ejecutó npm test sobre una copia limpia del backend (Node 22, ffmpeg 4.4, PGlite): 59 pruebas, 58 aprobadas y 1 fallida. La que falla es camara_medir.test.js (medirUna), que lee de ffmpeg la resolución y los cuadros por segundo y depende de cómo imprime ese dato la versión instalada de ffmpeg. No afecta a la API ni a la base de datos. Debe repetirse en la computadora del proyecto, donde antes se había ejecutado el conjunto, y decidir si el formato esperado se ajusta a las dos versiones.

10. Avance desde el Taller 8

El Taller 8 (7 de octubre, commit 09ecf67) dejó la arquitectura definida, el sistema con hub, nodos y aplicación, y la cámara como punto abierto. Entre esa fecha y el 21 de octubre se hicieron cambios en cuatro frentes.

Frente	Qué cambió	Commits
Configuración y emparejamiento (14 oct)	El Wi-Fi, el token y el servidor se escriben por comandos seriales y se guardan en memoria no volátil. Los nodos C3 se emparejan solos con el comando emparejar (ventana de 2 minutos).	adffdfa, 9f9c672
Cámara y puente (20 oct)	La cámara MC210 se conecta a un ESP32-S3 con NAT y WireGuard hacia el VPS. El ESP32 solo reenvía paquetes; ningún puerto de la cámara queda expuesto.	78f68d6, 26551bc
Detección por ONVIF (21 oct)	El backend se suscribe a los eventos de persona y movimiento que la cámara ya calcula, los guarda en eventos_camara y avisa con foto. Varias correcciones de robustez: cerrar la conexión tras cada petición, reintentar esperas cortadas por la cámara y tratar el cierre por inactividad.	002578c, 3ff3552, 935ea2f, c673839, 9281681, d06b989
Aplicación y cámara en vivo (21 oct)	La app muestra detecciones con fotos. Se ajustó la fluidez de la vista en vivo (fps configurable) y se agregó la medición de rendimiento por el túnel.	181abba, 39de23f

10.1 Configuración del puente: lo único que cambia al mover la cámara de red

La cámara se conecta a la red propia del ESP32-S3 (set_ap), que no cambia. Si el puente debe usar otro Wi-Fi para salir a Internet, solo se cambia el uplink con set_sta y se reinicia; el túnel y las reservas de IP no se tocan. Las claves se escriben en la consola serie y no se versionan.

```
set_sta <SSID_UPLINK> <CLAVE>            # Wi-Fi por el que el puente sale a Internet
set_ap  <SSID_PUENTE> <CLAVE_PUENTE>      # red donde se conecta SOLO la cámara (no
cambia)
dhcp_reserve add <MAC_MC210> 192.168.4.2  # IP fija de la cámara
set_vpn ...                               # línea generada por instalar_wireguard.sh
portmap add TCP 554  192.168.4.2 554  VPN   # RTSP por el túnel
portmap add TCP 2020 192.168.4.2 2020 VPN   # ONVIF por el túnel
restart
```

Explicación: el uplink y la red de la cámara son configuraciones independientes. Por eso se puede demostrar el sistema desde otra red (por ejemplo, el hotspot del teléfono) sin reconfigurar la cámara: el túnel WireGuard lo inicia el ESP32 hacia el VPS y mantiene la conexión con un keepalive de 25 segundos aunque haya un NAT en medio.

10.2 Decisión de la cámara: del módulo propio al puente

El Taller 8 dejaba abierto qué módulo implementaba la cámara y el firmware camara_ov5640 no había producido una imagen aceptable. Se adoptó una cámara MC210, que ya trae detección de personas y movimiento. El costo fue un enlace más (el puente), pero el resto del sistema no cambió, como preveía la arquitectura: la cámara sigue siendo un módulo que habla con el backend. Se conserva el firmware propio como alternativa.

10.3 Código clave: el motor de eventos de la cámara

Parte de la idea de que la cámara repite el mismo estado muchas veces. El motor guarda el último valor de cada detector y regla y solo emite un evento cuando cambia de verdad:

```
procesar(camaraId, m, ahoraMs) {
  const clave = `${m.detector}|${m.fuente || ''}|${m.regla || ''}`;
  const previo = c.claves.get(clave);

  if (previo === undefined) {
    c.claves.set(clave, { valor: m.valor, desde: ahoraMs });
    if (ahoraMs - c.inicioSyncMs <= this.ventanaSyncMs) return;   // sincronización: sin
alerta
    if (!m.valor) return;                                         // primer valor falso:
solo se anota
  } else if (previo.valor === m.valor) {
    c.repetidos++;                                                // repetido: se ignora
    return;
  } else {
    c.claves.set(clave, { valor: m.valor, desde: ahoraMs });
  }
  this.alEvento({ camaraId, detector: m.detector, valor: m.valor,
                  transicion: m.valor ? 'subida' : 'bajada', ... });
  if (m.valor) this._subida(camaraId, c, m, ahoraMs);              // decide aviso y
cooldown
}
```

Explicación: sin esta regla, cada repetición del mensaje Initialized de la cámara se contaría como un evento nuevo y llenaría la tabla y el teléfono de avisos. Con ella, solo cuentan los cambios reales false a true y true a false, el primer mensaje tras conectar se toma como sincronización y la política de alerta (solo con el sistema armado, retardo de salida y cooldown) se aplica después. Hay 14 pruebas de este motor en onvif_estados.test.js.

10.4 Código clave: configuración sin recompilar

Para no guardar secretos en el código, el concentrador lee sus parámetros de la memoria no volátil y los cambia por comandos seriales (wifi, token, servidor). Así el mismo binario sirve para cualquier vivienda y el repositorio no contiene claves.

10.5 Limitaciones y pendientes conocidos

- Vista en vivo: se detiene después de un tiempo. Hipótesis: ffmpeg termina con SIGKILL y deja abierta la sesión RTSP de la cámara, y no hay un vigilante que detecte el estancamiento. Pendiente: apagado ordenado (SIGINT y luego SIGKILL) y un watchdog; CAMARA_VIVO_FPS=15 aún debe fijarse en el .env del VPS.

- Pruebas de estabilidad del puente durante 24 horas y tras cortes de luz o de Wi-Fi: pendientes de hardware.
- Cifrado ESP-NOW desactivado de fábrica y MQTT previsto: se mantienen como mejoras de la versión alfa.
- Aviso de "cámara sin conexión", rotación de logs de Docker y retardo de entrada: propuestos, no implementados.
- Circuito de potencia de la sirena: diseñado y en simulación en el Taller 9; la simulación de conmutación del MOSFET quedó en depuración.

Anexo A. Evidencias por anexar antes de entregar

N.°	Evidencia	Cómo obtenerla
A1	Compilación sin errores de los tres programas	IDE de Arduino: captura de "Compilación terminada" de nodo_sensor, nodo_c3 y camara_ov5640.
A2	Servicio desplegado	Captura de https://sentinelhome.ddns.net/salud con ok: true.
A3	Registro y consulta con datos reales	Resultado de la consulta de la sección 7.3 y captura del historial en la app.
A4	Monitor serie del concentrador	Líneas [ESP-NOW] evento y [POST] codigo=201.
A5	Estados de carga, vacío y error en la app	Capturas del teléfono (con la sesión sin datos y con el servidor apagado).
A6	Pruebas PP-02, PP-05 y PP-07	Ejecutarlas y anotar el resultado en la tabla de la sección 9.4.
A7	npm test en la computadora del proyecto	Salida completa; confirmar si camara_medir.test.js pasa con su versión de ffmpeg.